

# Laporan Praktikum 1

Mata Kuliah Teknologi IoT II



**Oleh :**

Nama : Sofy Salsabila

NPM : 25782084

Kelas : TRI – 3C

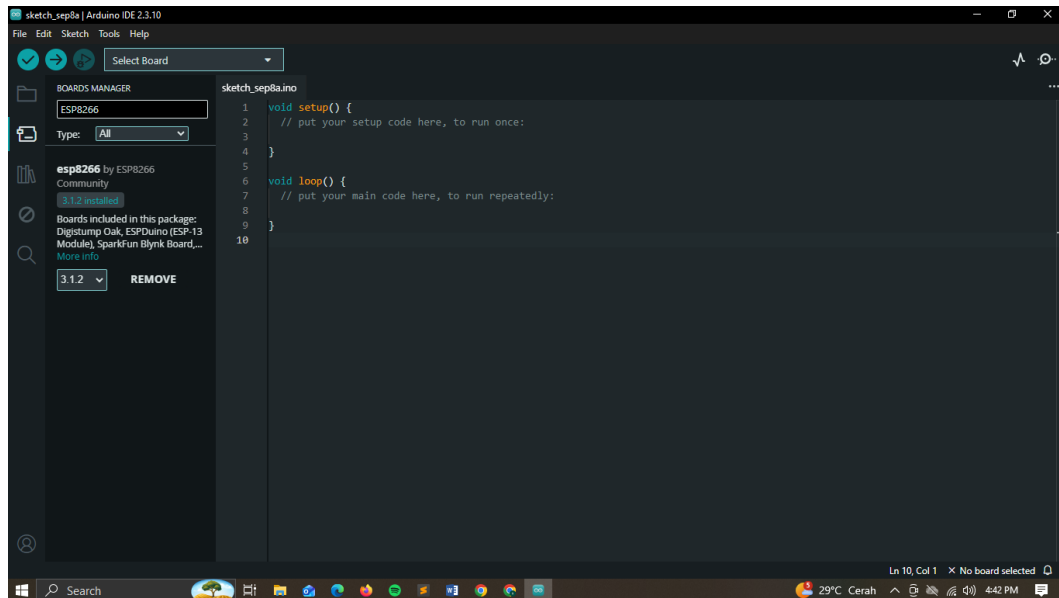
**Dosen Pengampu :**

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T., IPM. ASEAN Eng.

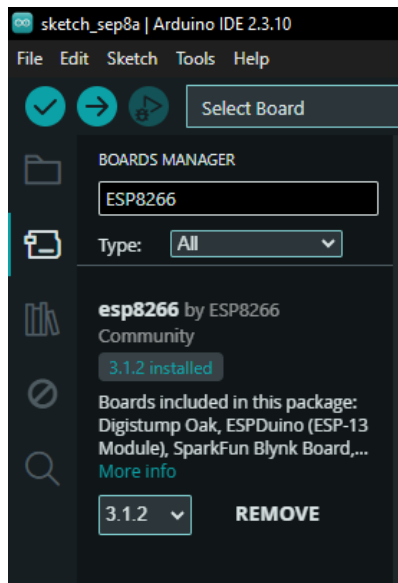
Agus Ambarwari, S.Pd, M.Kom

**PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET  
JURUSAN TEKNOLOGI INFORMASI  
POLITEKNIK NEGERI LAMPUNG  
2025/2026**

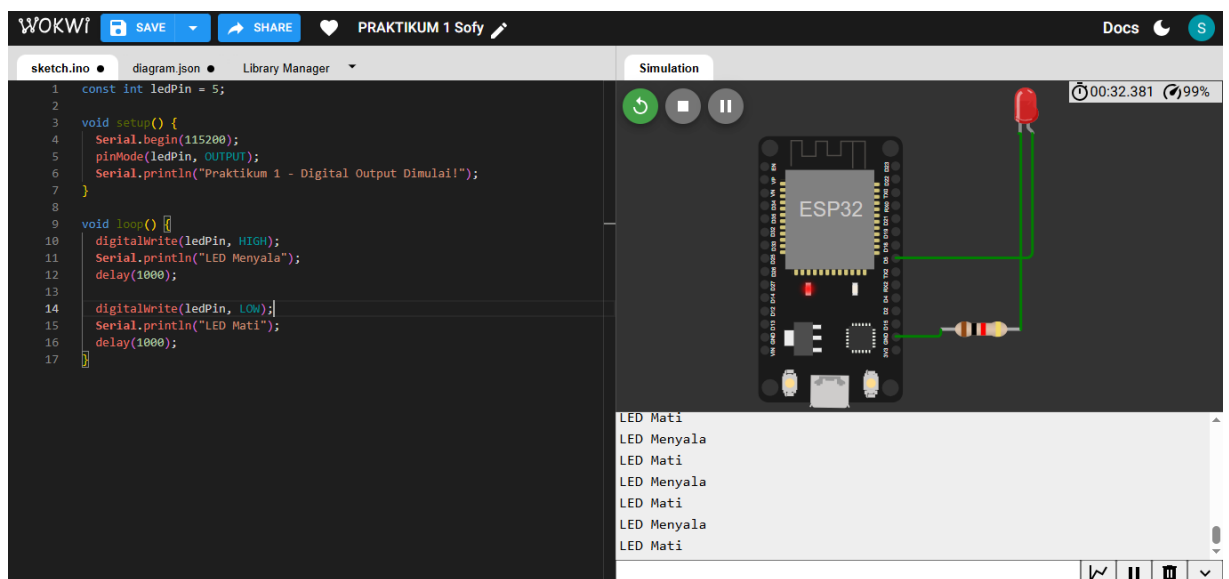
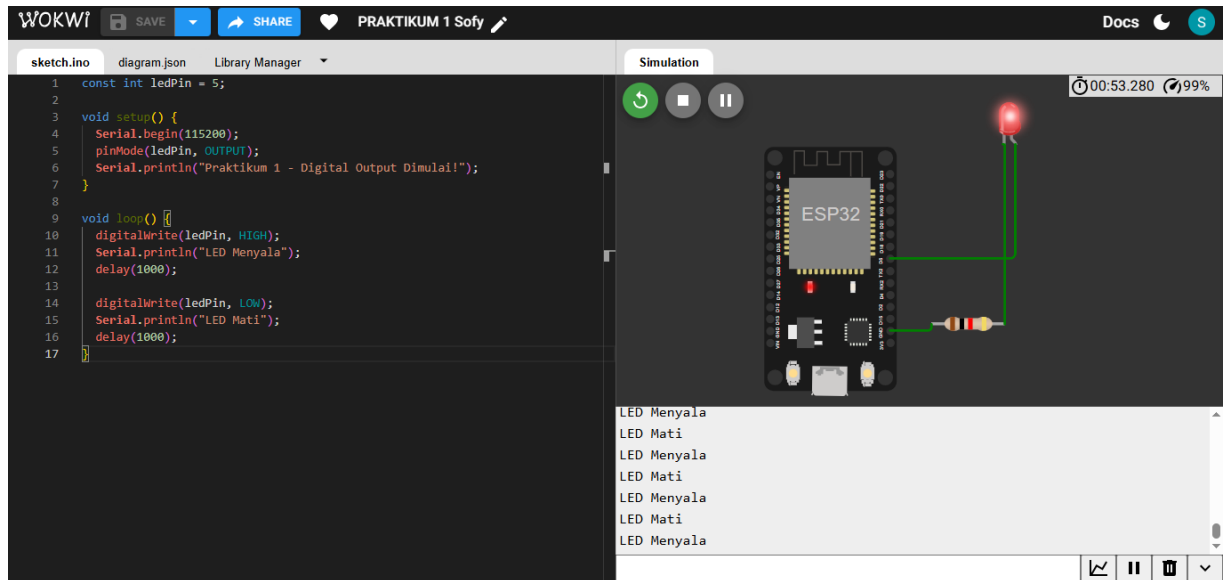
# Menginstall Arduino IDE



Setelah itu menginstall board package **ESP8266** pada perangkat lunak Arduino IDE



## Praktikum 1: *Digital Output* (Menyalakan LED)



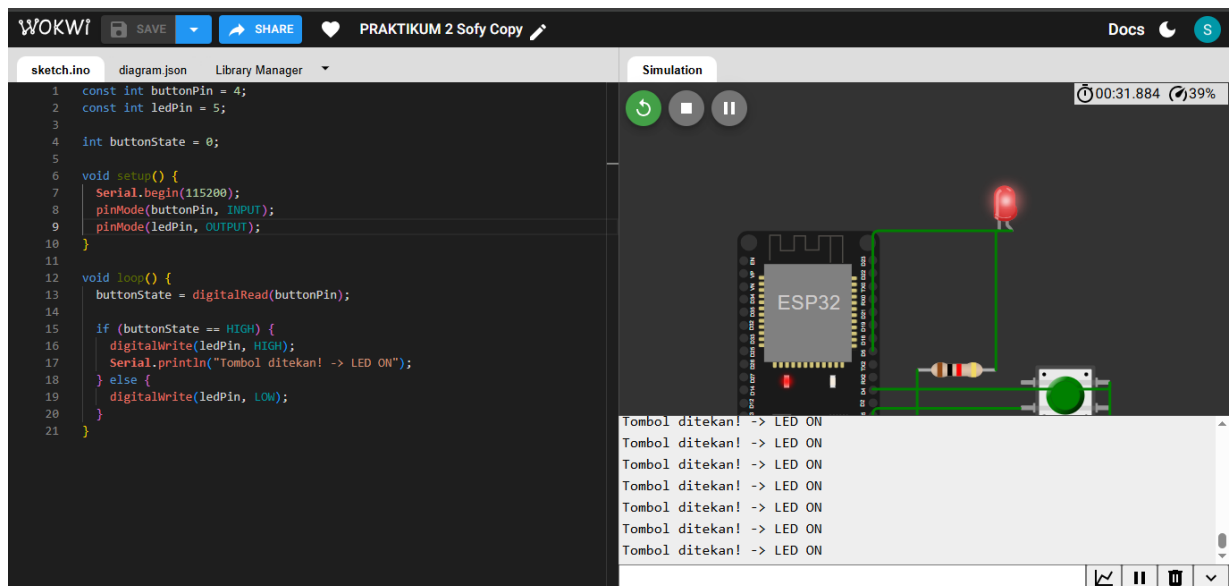
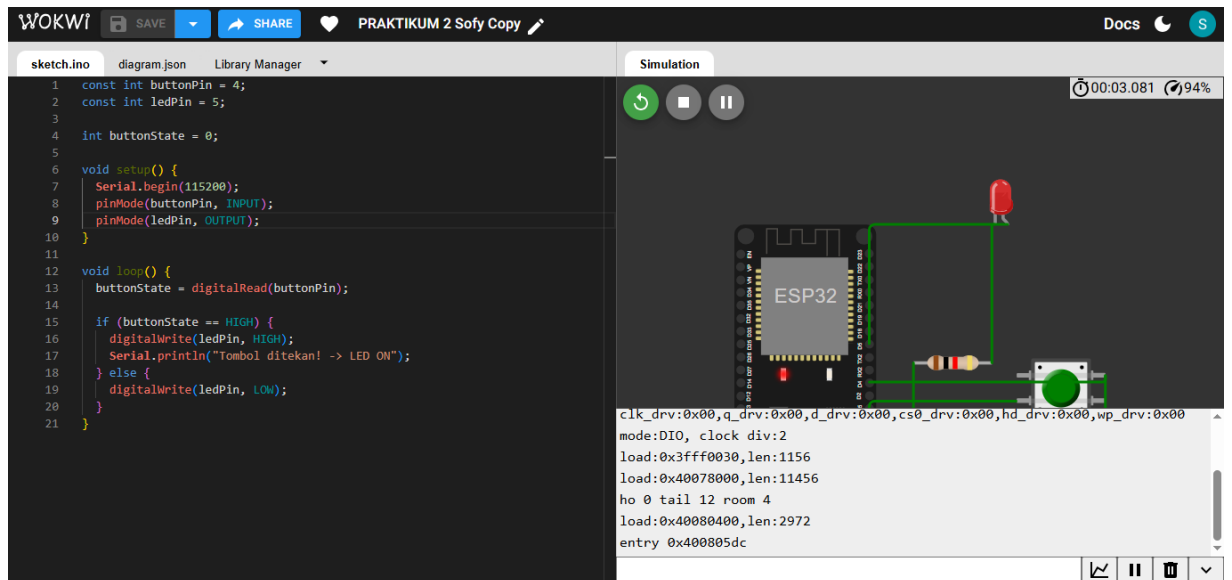
Praktikum 1 ini bertujuan memprogram pin digital D5 sebagai OUTPUT untuk menyalakan dan mematikan LED secara bergantian :

- **Saat HIGH** : Pin D5 mengeluarkan tegangan 3,3V sehingga arus mengalir, LED menyala, dan Serial Monitor mencetak "LED Menyala".
- **Saat LOW** : Tegangan diputus (0V) sehingga LED mati dan Serial Monitor mencetak "LED Mati".

Fungsi `delay(1000)` menahan masing-masing kondisi HIGH dan LOW selama 1 detik (1000 ms). Jadi, Karena ada perintah HIGH dan LOW yang dibungkus di dalam fungsi `loop()`, program akan mengeksekusi siklus "nyala 1 detik lalu mati 1

detik" secara berulang-ulang tanpa henti, sehingga menghasilkan efek LED berkedip yang selaras dengan teks di Serial Monitor.

## Praktikum 2: *Digital Input* (Membaca *Push Button*)



Praktikum 2 ini bertujuan membaca input digital dari push button menggunakan rangkaian resistor pull-down untuk mengendalikan LED secara interaktif

- **Kondisi LED Mati (Awal) :** Saat tombol tidak ditekan, pin D2 bernilai **LOW (0V)**. Blok else dieksekusi sehingga LED tetap mati.

- **Kondisi LED Menyala :** Saat tombol ditekan, arus 3,3V mengalir ke pin D2 sehingga `digitalRead()` bernilai **HIGH**. Kondisi `if (buttonState == HIGH)` terpenuhi, membuat LED menyala dan Serial Monitor mencetak "Tombol ditekan! -> LED ON".
- **Kondisi LED Kembali Mati :** Saat tombol dilepaskan, jalur listrik terputus dan pin D2 kembali bernilai **LOW (0V)**. Blok `else` dieksekusi kembali sehingga LED langsung padam.

## Latihan

- **Latihan 1 (Pemahaman Logika Pin):** Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

**Jawab :**

**Hasilnya :** NodeMCU akan mengalami **gagal booting** (boot failure), sehingga perangkat tidak bisa menyala secara normal dan program tidak akan berjalan.

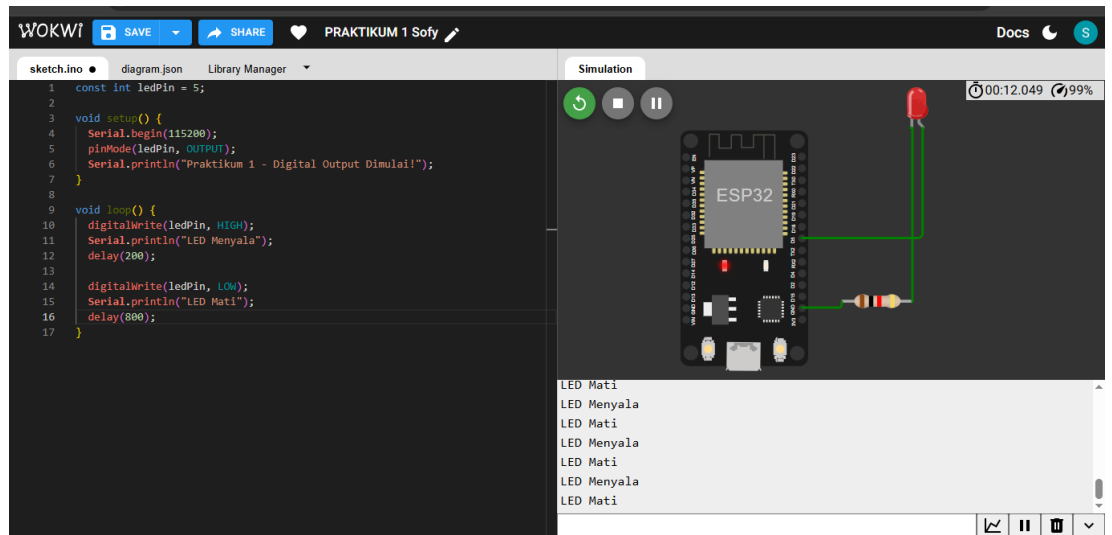
Jadi, sesuai pada tabel referensi pinout, pin D8 (GPIO15) bertuliskan "**Boot gagal jika ditarik HIGH**", yang artinya pin ini wajib bernilai **LOW** saat pertama kali diberi daya agar NodeMCU bisa booting dengan normal.

Namun, karena menggunakan rangkaian Active-High (*pull-down*), menekan tombol justru akan mengirimkan sinyal **HIGH**. Jika tombol ditekan tepat pas NodeMCU baru dinyalakan, pin D8 dipaksa menjadi **HIGH** sehingga syarat booting dilanggar. Akibatnya, proses booting sistem menjadi terkunci dan tidak dapat berjalan atau *dirunning*.

- **Latihan 2 (Implementasi Kode):** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.

**Jawab :**

Nilai `delay(200)` membuat LED menyala singkat selama 0,2 detik, sedangkan `delay(800)` membuat LED mati selama 0,8 detik. Perubahan ini menghasilkan efek kedipan LED yang tidak simetris (asymmetric blinking).



- **Latihan 3 (Analisis Sirkuit):** Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

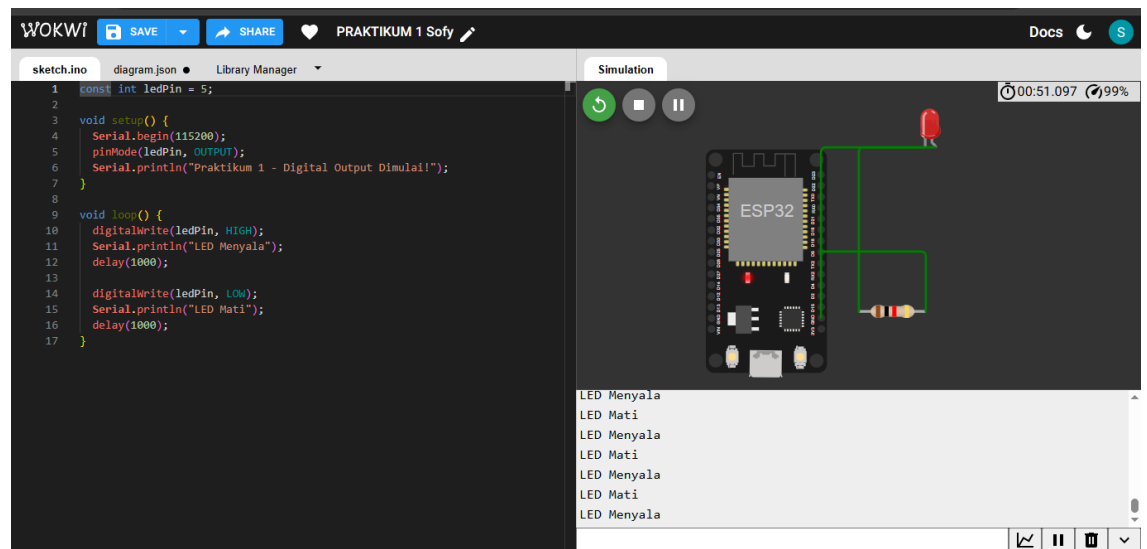
**Jawab :**

Program tidak mengalami *error* kompilasi. Kode program tetap berhasil di compile (compile success) dan simulasi berjalan tanpa *error*. Hal ini karena compiler cuma bertugas mengecek penulisan kodenya (syntax) sudah benar atau belum, bukan mengecek kabel atau komponen yang terpasang di sirkuit.

Kemudian, LED tidak menyala karena LED adalah komponen dioda yang bersifat pintu satu arah buat arus listrik, jadi dia punya kutub positif (Anoda) dan negatif (Katoda):

- Jika, anoda terhubung ke pin positif **D5 / D1** dan Katoda ke GND. Pintunya terbuka, arus listrik mengalir dari pin ke GND, sehingga LED menyala.
- Jika, anoda terhubung ke GND dan Katoda ke pin **D5 / D1**. Pintunya jadi terkunci, jadi LED menahan atau menyumbat aliran listrik. Karena tidak ada arus yang bisa mengalir ke GND, LED tetap mati.

Karena disini saya memakai ESP32, jadi tidak ada D1, maka saya disini memakai D5.



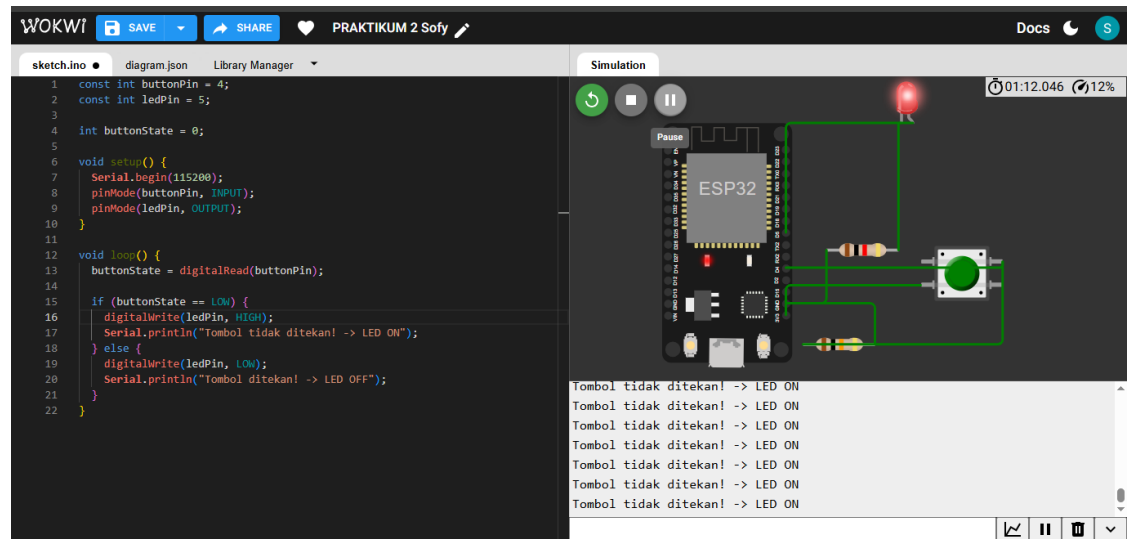
- **Latihan 4 (Pengembangan Algoritma):** Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

**Jawab :**

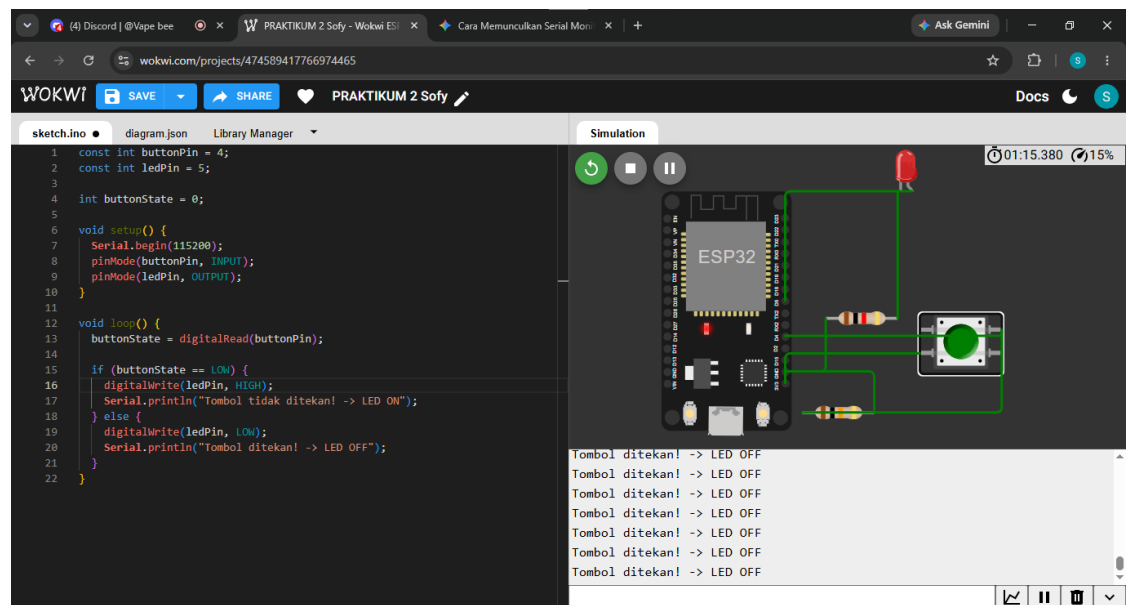
Jadi disini saya mengubah logika pada blok **if-else** diubah dengan memeriksa kondisi **buttonState == LOW** terlebih dahulu.

- Pada saat tombol tidak ditekan (**buttonState == LOW**):  
Program akan menjalankan perintah `digitalWrite(ledPin, HIGH)` sehingga LED ON (menyala).
- Pada saat tombol ditekan (**buttonState == HIGH**):  
Program masuk ke blok else dan menjalankan perintah `digitalWrite(ledPin, LOW)` sehingga LED OFF (Mati)

Ketika tombol button di tidak pencet, **LED ON (LED MENYALA)**



Ketika tombol button di pencet **LED OFF (MATI)**

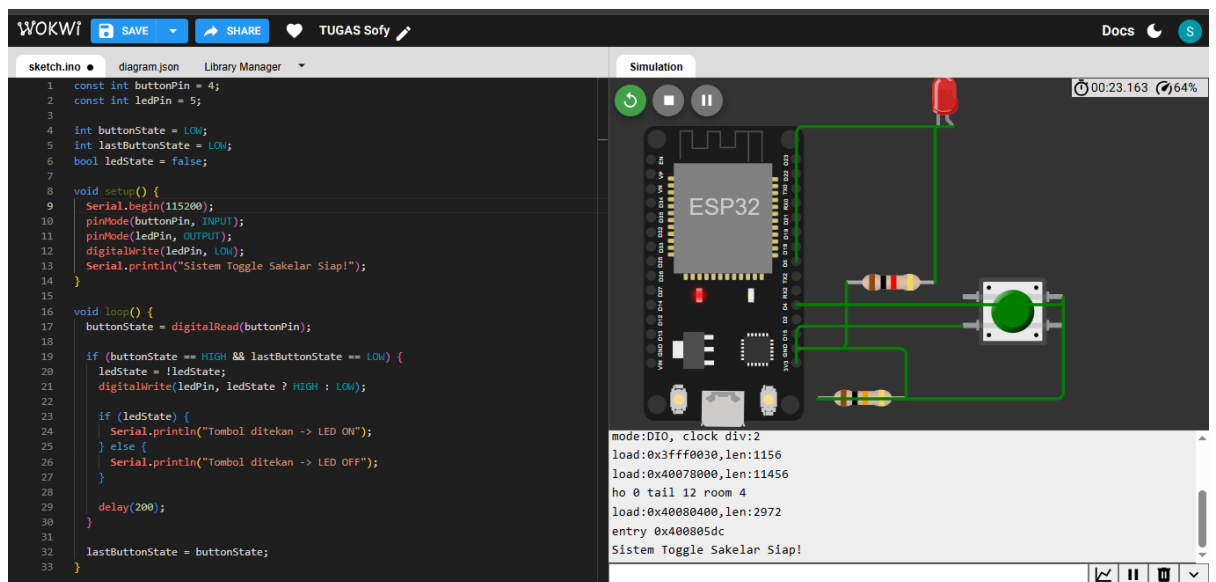




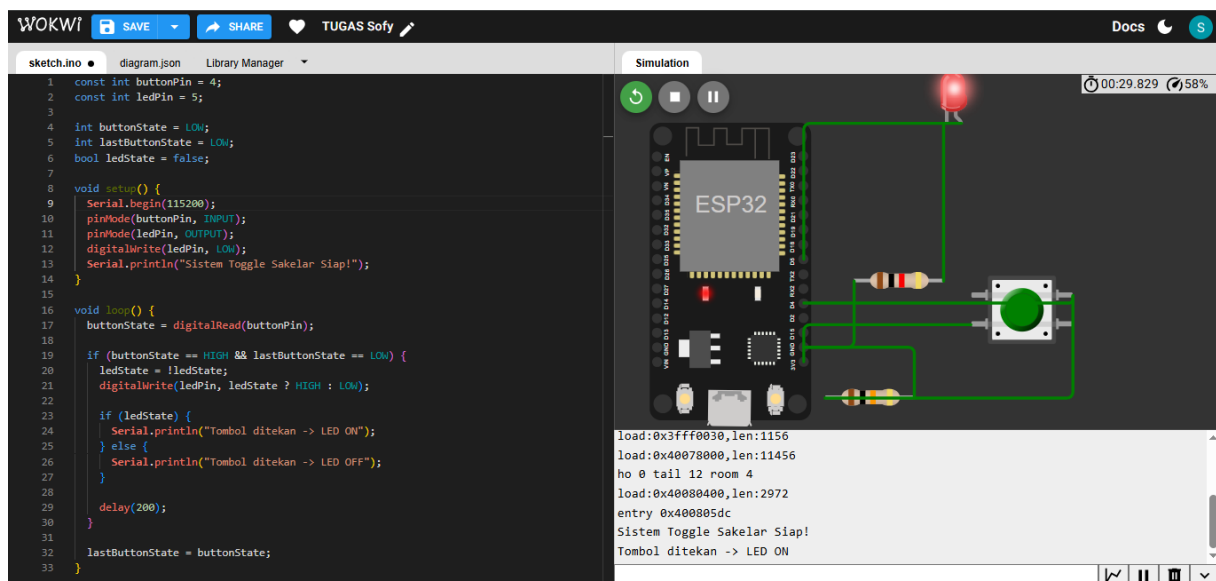
# TUGAS

## Modifikasi Sistem Sakelar *Toggle (Latching)*

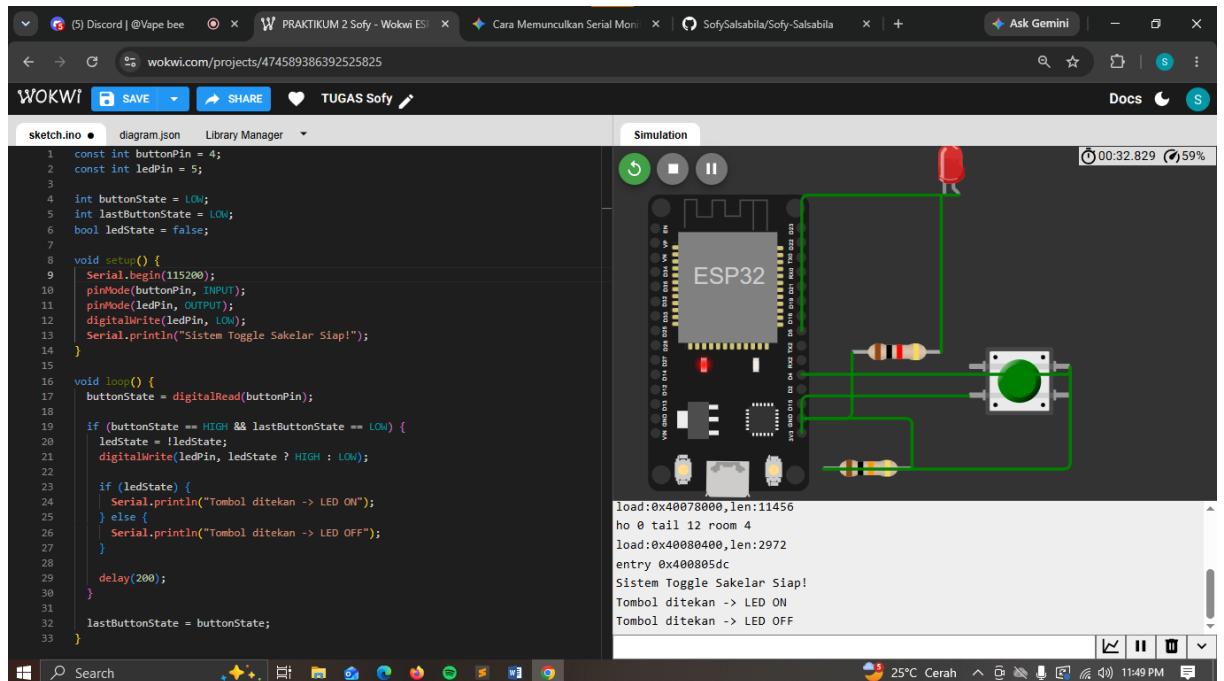
Ketika dijalankan atau di running, menampilkan bahwa LED nya **siap** untuk digunakan



Ketika dipencet tombol button, LED akan ON atau **MENYALA**



Ketika tombol button dipencet kembali, **LED** akan **OFF** atau **MATI**



Tugas atau praktikum ini bertujuan agar tombol biasa bisa berfungsi seperti sakelar lampu rumah di ESP32. Saat pertama dinyalakan, pin 4 disiapin buat baca tombol, pin 5 buat menyalakan LED yang posisinya masih mati, dan Serial Monitor bakal menampilkan pesan status awal yaitu “Sistem Toggle Sakelar siap!” yang berarti sistem siap untuk digunakan atau di pencet tombolnya.

Ketika tombol ditekan, program akan mendeteksi ketika tombolnya baru mulai dipencet, lalu otomatis mengubah status LED kalau tadinya mati jadi **menyala (on)**, dan kalau lagi menyala jadi **mati (off)**. Nantinya, ada jeda singkat 200 milidetik (**delay(200)**) biar sinyal dari tombol tetap stabil dan LED nya tidak kedip-kedip.